Generating algorithms for multivariate multicycle codes

F. A. Mian, O. Gwilliam, and S. Krastanov

Pseudocode accompanying "Multivariate Multicycle Codes for Complete Single-Shot Decoding" (formerly Appendix A of the manuscript).

In this appendix, we present Algorithm 1 for constructing Multivariate Multicycle (MM) codes via Koszul complexes over quotient rings. The algorithm takes polynomials $F_1, \ldots, F_t$ in the quotient ring

$$S = \mathbb{F}_2[x_1, \ldots, x_D]/\langle x_1^{\ell_1} - 1, \ldots, x_D^{\ell_D} - 1\rangle, \qquad (1)$$

where $t \geq 2$ is the number of polynomials, $D \geq 1$ is the number of variables, and $\ell_1, \ldots, \ell_D \in \mathbb{Z}_{\geq 1}$ are exponents defining the ideal $I = \langle x_1^{\ell_1} - 1, \ldots, x_D^{\ell_D} - 1\rangle$. It then constructs the Koszul complex $K_\bullet([F_1, \ldots, F_t]; S)$ over S, where each boundary map ∂_k is a matrix with entries in S. Each entry is expanded in the regular representation of the finite abelian group. The resulting binary blocks are multilevel circulant matrices; for one cyclic factor they are ordinary circulant matrices.

Throughout the main text, ∂_k denotes the matrix of the boundary map acting on column vectors, so that $P_X = \partial_q$, $P_Z = \partial_{q+1}^\top$, $M_X = \partial_{q-1}$ and $M_Z = \partial_{q+2}^\top$. `Oscar.jl` represents module homomorphisms by matrices acting on row vectors; the array $\partial[k]$ in Algorithms 1 and 2 therefore stores the transpose $\partial_k^\top$, which is why the assignments there read $P_X \leftarrow \partial[q]^\top$ and $P_Z \leftarrow \partial[q+1]$. Algorithm 2 presents an alternative "symbolic" approach: it constructs the Koszul complex over the polynomial ring $\mathbb{F}_2[y_1, \ldots, y_t]$ with standard boundary maps having entries in $\{0, y_1, \ldots, y_t\}$, then substitutes each y_i with the circulant matrix corresponding to $F_i \in S$. Both methods employ Algorithm 3 to convert polynomials to circulant matrices. After conversion, we verify pairwise commutativity of the resulting matrices. The algorithms are implemented in the open-source `QuantumClifford.jl` library using the OSCAR computer algebra system [OSCAR] and the Julia programming language.

Algorithm 1: MM code via Koszul complex over quotient ring

Input: $\ell_1, \ldots, \ell_D$, $F_1, \ldots, F_t \in S = \mathbb{F}_2[x_1, \ldots, x_D]/\langle x_i^{\ell_i} - 1, \cdots x_D^{\ell_D} - 1\rangle$
Output: P_X, P_Z, M_X, M_Z

$s \leftarrow \prod_{i=1}^D \ell_i, t \leftarrow \text{length}(F_1, \ldots, F_t)$
for $i \leftarrow 1$ **to** t **do**
  $C_i \leftarrow \text{PolynomialToCirculant}(F_i, \ell_1, \ldots, \ell_D)$
end
`// Check pairwise commutativity`
for $i \leftarrow 1$ **to** t **do**
  for $j \leftarrow i+1$ **to** t **do**
    assert $C_i C_j \equiv C_j C_i \pmod 2$
  end
end
`// Construct Koszul complex over S`
$K \leftarrow \text{KoszulComplex}([F_1, \ldots, F_t], S)$
$\partial \leftarrow \text{Array}(t)$
for $k \leftarrow 1$ **to** t **do**
  $\partial_k \leftarrow \text{map}(K, k)$ $B_k \leftarrow \text{matrix}(\partial_k)$
  $(r, c) \leftarrow \text{size}(B_k)$
  $M_k \leftarrow \mathbf{0}_{rs \times cs}(\mathbb{Z})$
  for $i \leftarrow 1$ **to** r **do**
    for $j \leftarrow 1$ **to** c **do**
      $f \leftarrow B_k[i, j]$
      if $f \neq 0$ **then**
        $C \leftarrow \text{PolynomialToCirculant}(f, \ell_1, \ldots, \ell_D)$
        $M_k[(i-1)s+1 : is, (j-1)s+1 : js] \leftarrow C$
      end
    end
  end
  $\partial[k] \leftarrow M_k \bmod 2$
end
for $k \leftarrow 2$ **to** t **do**
  assert $\partial[k] \cdot \partial[k-1] \equiv 0 \pmod 2$
end
if $t = 2$ **then**
  $P_X \leftarrow \partial[1]^\top$, $P_Z \leftarrow \partial[2]$
  $M_X \leftarrow \varnothing$, $M_Z \leftarrow \varnothing$
else
  $q \leftarrow \lfloor t/2 \rfloor$
  $P_X \leftarrow \partial[q]^\top$, $P_Z \leftarrow \partial[q+1]$
  if $t \geq 4$ **then**
    $M_X \leftarrow \partial[q-1]^\top$
  end
  if $t \geq 3$ **then**
    $M_Z \leftarrow \partial[q+2]$
  end
end
assert $P_X P_Z^\top \equiv 0 \pmod 2$
if $M_X \neq \varnothing$ **then**
  assert $M_X P_X \equiv 0 \pmod 2$
end

Algorithm 2: MM code via Koszul complex over polynomial ring

Input: $\ell_1, \ldots, \ell_D$, $F_1, \ldots, F_t \in R = \mathbb{F}_2[x_1, \ldots, x_D]/\langle x_i^{\ell_i} - 1, \cdots x_D^{\ell_D} - 1\rangle$
Output: P_X, P_Z, M_X, M_Z

$s \leftarrow \prod_{i=1}^{D} \ell_i, t \leftarrow \text{length}(F_1, \ldots, F_t)$
for $i \leftarrow 1$ **to** t **do**
  $C_i \leftarrow \text{PolynomialToCirculant}(F_i, \ell_1, \ldots, \ell_D)$
end
`// Check pairwise commutativity`
for $i \leftarrow 1$ **to** t **do**
  for $j \leftarrow i+1$ **to** t **do**
    assert $C_i C_j \equiv C_j C_i \pmod 2$
  end
end
`// Construct Koszul complex boundary maps`
$R, y \leftarrow \mathbb{F}_2[y_1, \ldots, y_t]$
$K \leftarrow \text{KoszulComplex}(y)$
$\partial \leftarrow \text{Array}(t)$
for $k \leftarrow 1$ **to** t **do**
  $B_k \leftarrow \text{matrix}(\text{map}(K, k))$ $(r, c) \leftarrow \text{size}(B_k)$
  $\partial_k \leftarrow \mathbf{0}_{rs \times cs}(\mathbb{Z})$
  for $i \leftarrow 1$ **to** r **do**
    for $j \leftarrow 1$ **to** c **do**
      if $B_k[i, j] = y_m$ **then**
        $\partial_k[(i-1)s+1 : is, (j-1)s+1 : js] \leftarrow C_m$
      end
    end
  end
  $\partial[k] \leftarrow \partial_k \bmod 2$
end
for $k \leftarrow 2$ **to** t **do**
  assert $\partial[k] \cdot \partial[k-1] \equiv 0 \pmod 2$
end
if $t = 2$ **then**
  $P_X \leftarrow \partial[1]^\top$, $P_Z \leftarrow \partial[2]$
  $M_X \leftarrow \varnothing$, $M_Z \leftarrow \varnothing$
else
  $q \leftarrow \lfloor t/2 \rfloor$
  $P_X \leftarrow \partial[q]^\top$, $P_Z \leftarrow \partial[q+1]$
  if $t \geq 4$ **then**
    $M_X \leftarrow \partial[q-1]^\top$
  end
  if $t \geq 3$ **then**
    $M_Z \leftarrow \partial[q+2]$
  end
end
assert $P_X P_Z^\top \equiv 0 \pmod 2$
if $M_X \neq \varnothing$ **then**
  assert $M_X P_X \equiv 0 \pmod 2$
end
if $M_Z \neq \varnothing$ **then**
  assert $M_Z P_Z \equiv 0 \pmod 2$
end
return (P_X, P_Z, M_X, M_Z)

Algorithm 3: PolynomialToCirculant

Input: Polynomial F, orders $\ell_1, \ldots, \ell_D$
Output: Multilevel circulant matrix (circulant for $D = 1$) $C \in \mathbb{F}_2^{s \times s}$ s.t. $s = \prod_{i=1}^{D} \ell_i$

$s \leftarrow \prod_{i=1}^{D} \ell_i$
$C \leftarrow \mathbf{0}_{s \times s}(\mathbb{F}_2)$
$f \leftarrow \text{lift}(F)$
for $j \leftarrow 0$ **to** $s-1$ **do**
  $tmp \leftarrow j$
  for $k \leftarrow D$ **downto** 1 **do**
    $idxs[k] \leftarrow tmp \bmod \ell_k$
    $tmp \leftarrow \lfloor tmp/\ell_k \rfloor$
  end
  for $term \in terms(f)$ **do**
    $c \leftarrow \text{coeff}(term, 1)$
    if $c = 0$ **then**
      continue
    end
    $e_k \leftarrow \text{degree}(term, k)$ for $k = 1, \ldots, D$
    $row \leftarrow 0$
    for $k \leftarrow 1$ **to** D **do**
      $row \leftarrow row \cdot \ell_k + ((idxs[k] + e_k) \bmod \ell_k)$
    end
    $C[row+1, j+1] \leftarrow C[row+1, j+1] + c$
  end
end
return C